CentraleSupelec **Computational Fluid Dynamics**

Project I
Resolution of elliptic partial differential equations
– Pressure field in an incompressible flow –

Key concepts

- Resolution of an elliptic PDE such as a steady diffusion problem involving a Laplace or Poisson equation
- Iterative methods for solving linear systems of equations

1 Learning with AI

The course objective is to train students in numerical methods, i.e. theory, programming, and review of CFD codes.

AI is fantastic for self-learning and productivity. It can also be a pitfall for students if you don't analyse the code with a critical eye and just take it for granted. Reasons:

- Small personal value in blindly copying/pasting/launching a code you could not write and that you don't understand
- You remember only concepts and implementation details when putting in some effort to achieve the results. Getting code easily is less rewarding, imprinting the developed skills less deeply.
- Code developed in an MSc course is standard and can nowadays be generated flawlessly, making engineers unaware of real-life CFD programming experience in a large, complex code.

Our usage of AI in this course:

- Learn to decompose a problem into simple tasks and leverage AI for code generation of well-defined tasks
- Make sure you understand the entire code, and edit and adapt it yourself
- Learn and implement advanced methods with AI

Of course, modern LLMs can do all that for you at once. However, this course is about training you while acknowledging the benefit of AI in day-to-day engineering tasks.

Throughout this document, the **AI track** boxes give you pedagogical guidelines on how to combine the three steps above with AI assistance on a specific question. In this first project, the prompts are given to you, so that you can experience the full workflow at least once before flying on your own. In the next project, you will be asked to write your own prompts, guided by instructions. In the final project, nothing will be said: it will be up to you to decide how, when, and whether to call on AI.

AI usage is authorized and encouraged throughout this course. What matters is not how you got there: you will be evaluated on your code, which you must master entirely and be able to explain, justify, and adapt on the spot.

In companies or laboratories, only use LLMs provided by your employer, and never share proprietary code online.

2 Pressure field in a steady incompressible flow

We look for the pressure field of the specified velocity field in part A:

$$u(x,y) = -U\sin(\pi x/L)\cos(\pi y/L) \tag{1}$$
$$v(x,y) = U\cos(\pi x/L)\sin(\pi y/L) \tag{2}$$

with $x, y \in [0, L]$, $L = 0.3$ m and $U = 0.1$ m/s. The fluid density ρ_0 is 1.2 kg/m^3.

Taking the divergence of the Navier-Stokes equations yield the following partial derivative equation for the pressure field:

$$\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = -\rho_0 \left[\left(\frac{\partial u}{\partial x}\right)^2 + 2\frac{\partial u}{\partial y}\frac{\partial v}{\partial x} + \left(\frac{\partial v}{\partial y}\right)^2 \right] = f(x,y) \tag{3}$$

The pressure field follows then a Poisson equation (a common elliptic PDE) with the right-hand-side given here explicitly for the specified velocity field:

$$f(x,y) = \rho_0 \left(\frac{\pi U}{L}\right)^2 \left(\cos\left(\frac{2\pi x}{L}\right) + \cos\left(\frac{2\pi y}{L}\right) \right) \tag{4}$$

As for boundary conditions,

- The pressure field obeys homogeneous Neumann conditions $\left(\dfrac{\partial p}{\partial n} = 0\right)$ at the right and top boundaries
- Pressure values are fixed at the bottom and left boundaries:

$$p(x=0,y) = g(y) = \frac{1}{4}\rho U^2 \left(1 - \cos(2\pi y/L)\right)$$
$$p(x,y=0) = g(x) = \frac{1}{4}\rho U^2 \left(1 - \cos(2\pi x/L)\right)$$

The two-dimensional pressure field $p(x, y)$ in $\Omega = [0, L]^2$ is thus given by the following equation:

$$\begin{aligned} \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} &= f(x,y) \quad \text{for } \mathbf{x} \in \text{int}(\Omega), \\ \frac{\partial p}{\partial n} &= 0 \quad \text{for } \mathbf{x} \in \partial\Omega_{top} \cup \partial\Omega_{right} \\ p(0,y) &= g(y) \quad \text{for } \mathbf{x} \in \partial\Omega_{left} \\ p(x,0) &= g(x) \quad \text{for } \mathbf{x} \in \partial\Omega_{bottom} \end{aligned} \tag{5}$$

3 Solution of the Poisson equation

3.1 Numerical resolution

a) Implement the Jacobi method to solve Eq. (5) for $N_x = 31$ mesh cells along each side and $N_t = 1000$ iterations. Plot the corresponding pressure field.

AI track

1. **Simplify the task and prompt AI**
   - How can I make the problem simpler? Not just for the LLM, but also for you to grasp the code.

   Prompt used:

   ```
   Provide me with Python code to solve the Laplace equation using the
   Jacobi method.  For the time being, limit optimizations that might
   hinder the readability and understanding of the code, particularly in
   the core of the numerical scheme.  The theoretical formulas should be
   easily recognizable in the code.
   ```

2. **Code review and skills-building**
   Without any help from AI, review the code and make sense of each line. Adapt and correct the code when needed. Clarify the documentation and code to your liking, and don't leave any part unclear to you. Remove unneeded or unrequested code features.

3. **Assembling the original problem**
   Make incremental changes (yourself or with AI) to reach the desired objectives, while repeating task 2.
   Here, the gap is small enough to ask the LLM to fill it.
   Prompt used:

   ```
   Now, read only section 2 of this pdf file, adapt the code to the
   problem setup described there, that is, include the source term in
   the Poisson equation and the correct boundary conditions.
   ```

   Check and clean the code.

b) The exit of the iterative procedure will now be driven by an error control at each iteration with respect to Eq. (5), which is known as the residual norm $||r|| = ||b - Ax||$ where $Ax = b$ is the discretized Poisson equation. The iterations are stopped when the initial residual has been reduced by several orders of magnitude: $||r|| < \varepsilon||r_0||$. ε is a user-specified accuracy threshold. Update your Jacobi algorithm correspondingly with $\varepsilon = 10^{-3}$. Check the velocity field and plot the evolution of the residual norm $||r||/||r_0||$ along the Jacobi iterations in a semi-logarithmic plot.

AI track
Learning
If you are unfamiliar with the concept of residual, ask the LLM a few questions about it in a new, independent request:

- What is the residual of the solution of a PDE?
- Why is it useful to compute it?
- How is it computed for a discretized Poisson equation?

Make sure you connect this with the course contents, and do not hesitate to ask the teachers in the room.
Then answer question b) by implementing the residual control yourself.

c) Repeat the calculation for $N_x = 101$ and explain why the Jacobi method becomes slow on finer meshes.

d) Implement the Gauss-Seidel and SOR methods (still controlled with $\varepsilon = 10^{-3}$). Compare the three methods with respect to the obtained solution, number of iterations, and computational time, and comment.

> ~~**AI track**~~
> **Learning**
> Understanding the differences in implementation between the Jacobi, Gauss-Seidel and SOR methods is important: implement all three yourself, without any LLM.

e) Optimize your code and repeat question d). Comment.

> **AI track**
> **Optimization**
> After a few optimization attempts of your own (leave comments for context), ask the LLM to optimize your code, learning from the tricks it proposes. When an aggressive optimization hurts code readability (a personal judgment call), find a balanced compromise so that you can always modify and adapt the code yourself later.
> **This matters in this course, where we require you to be accountable for every single line.**

f) The pressure force applied to the right side is given by $F = \int_{[0,L]} p(x, y)\, dy$ and is directly proportional to the average $\langle p \rangle = \dfrac{1}{L} \int_{[0,L]} p(x, y)\, dy$. Compute $\langle p \rangle$ for one of the obtained pressure fields and identify the different numerical errors that can affect your result.

> **AI track**
> Create a function to compute $\langle p \rangle$ from the pressure array. As always, review, understand, and clean the code.

g) By running your program on finer meshes, check that the previous SOR algorithm seen in the course textbook is second-order accurate.

> **AI track**
> Ask the LLM to wrap your code into a function (if not done already) so that you can easily loop over several meshes.

h) Report a reliable value of $\langle p \rangle$ while quantifying its accuracy.

3.2 Comparison with analytical solution

Let's first define the pressure coefficient field:

$$C_p = \frac{p - p_0}{\frac{1}{2}\rho_0 U^2} \tag{6}$$

where p_0 is a reference pressure value, here chosen as null.

In a general case, you won't access the exact error of our simulation since we cannot put our hands on the exact solution of our problem. However, we can always compute the residual, hence the interest in such a quantity that can always indicate how far we are from solving exactly our

problem. In this specific case, we have the exact analytical solution

$$C_p^{exact} = \frac{1}{2}\left(2 - \cos\left(\frac{2\pi x}{L}\right) - \cos\left(\frac{2\pi y}{L}\right)\right). \tag{7}$$

i) For a given mesh, compare the evolution of the error $||C_p^{num} - C_p^{exact}||/||C_p^{exact}||$ along the iterations with the residual norm evolution. Comment.

j) Identify and comment the role of residual plots and relaxation factors in Ansys Fluent (when solving steady flows).

3.3 Advanced methods

The course textbook introduces several advanced methods, and many others exist that are not covered there.

> **AI track**
> Discuss possible improvements with the LLM, then implement one advanced method for solving the Poisson equation.

4 Extra

a) Propose and detail a new numerical method which would yield fourth-order accuracy for solving the Poisson equation.

b) In the list of PDEs below, identify their classification (hyperbolic/parabolic/elliptic) and, when elliptic, write the corresponding Jacobi updating formula:

- 2D Unsteady heat equation:

$$\frac{\partial T}{\partial t} = a\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right)$$

- Fully developed flow along z-axis with *uniform* pressure:

$$-\frac{dp}{dz} + \mu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = 0$$

- Wave equation:

$$\frac{\partial^2 u}{\partial t^2} - c^2\frac{\partial^2 u}{\partial x^2} = 0$$

- 3D Steady heat equation:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2} = 0$$

- 2D Steady advection:

$$u(x,y)\frac{\partial T}{\partial x} + v(x,y)\frac{\partial T}{\partial y} = 0$$

- 3D Steady heat transfer development in a duct ($u(x,y) > 0$):

$$u(x,y)\frac{\partial T}{\partial x} = a\left(\frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2}\right)$$

- 2D steady advection and diffusion:

$$u(x,y)\frac{\partial T}{\partial x} + v(x,y)\frac{\partial T}{\partial y} = a\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right)$$

- Burgers equation:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}$$

- 2D Steady heat equation with temperature-sensitive thermal conductivity:

$$\frac{\partial}{\partial x}\left(\lambda(T)\frac{\partial T}{\partial x}\right) + \frac{\partial}{\partial y}\left(\lambda(T)\frac{\partial T}{\partial y}\right) = 0$$